EXP 10: IMPLEMENT EXPECTATION AND MAXIMATION ALGORITHM

NAME: SAMUEL I

REG NO:192312584

AIM:

To implement the Expectation-Maximization (EM) algorithm in Python for clustering data using a Gaussian Mixture Model.

PROCEDURE

1. Import the required libraries and load the dataset into the program.
2. Create a Gaussian Mixture Model with the required number of clusters.
3. Train the model using the Expectation-Maximization algorithm.
4. Predict the cluster labels for the dataset.
5. Display the predicted labels and accuracy to evaluate the clustering performance.

PROGRAM:

```
# Expectation-Maximization (EM) Algorithm using Gaussian Mixture Model

from sklearn.datasets import load_iris
from sklearn.mixture import GaussianMixture
from sklearn.metrics import accuracy_score
from scipy.stats import mode
import numpy as np

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
```

```
# Create EM model
gmm = GaussianMixture(n_components=3, random_state=42)


# Train model
gmm.fit(X)


# Predict clusters
clusters = gmm.predict(X)


# Map clusters to actual class labels
labels = np.zeros_like(clusters)
for i in range(3):
    labels[clusters == i] = mode(y[clusters == i], keepdims=True).mode[0]


# Display results
print("Predicted Labels:")
print(labels)


print("\nAccuracy:")
print(accuracy_score(y, labels) * 100, "%")
```

OUTPUT:

```
Predicted Labels:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 2 1 2 1
 1 1 1 2 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2]

Accuracy:
96.66666666666667 %
```

2.
PROGRAM:

```
from sklearn.datasets import load_iris
from sklearn.mixture import GaussianMixture

X, y = load_iris(return_X_y=True)

gmm = GaussianMixture(n_components=3, random_state=42)
gmm.fit(X)

labels = gmm.predict(X)

print("Cluster Labels:")
print(labels)

print("\nCluster Means:")
print(gmm.means_)

print("\nLog-Likelihood:", gmm.score(X))
```

OUTPUT:

```
Cluster Labels:
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 0 2 0 2 0 2
 2 2 2 0 2 2 2 2 2 0 2 2 2 2 2 2 2 2 2 2 2 2 2 2 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0]

Cluster Means:
[[6.54639415 2.94946365 5.48364578 1.98726565]
 [5.006      3.428      1.462      0.246     ]
 [5.9170732  2.77804839 4.20540364 1.29848217]]

Log-Likelihood: -1.2013110850984177
```

3.
PROGRAM:

```
from sklearn.datasets import load_breast_cancer
from sklearn.mixture import GaussianMixture

X, y = load_breast_cancer(return_X_y=True)

gmm = GaussianMixture(n_components=2, random_state=42)
gmm.fit(X)

labels = gmm.predict(X)
```

```
print("Cluster Labels:")
print(labels)

print("\nMeans:")
print(gmm.means_)

print("\nLog-Likelihood:", gmm.score(X))
```

OUTPUT:

```
Cluster Labels:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 1 1 0 1 1 0 0 0 0 1 0 1 1 1 1 1 0 0 1 0 0 1 1 1 1 0 1 0 0 1 1 1 1 0 1 0 1
 1 0 1 0 0 1 1 0 0 0 1 0 1 0 1 1 1 1 1 1 1 0 0 1 1 1 1 0 1 1 1 1 0 1 1 0 1 1
 1 1 1 1 1 1 1 0 0 0 1 0 0 1 1 1 0 0 1 0 1 0 0 1 0 1 1 1 0 1 1 0 1 1 1 1 1 0 1
 1 1 1 1 1 1 1 1 1 0 0 1 1 1 0 0 1 0 1 1 0 0 1 1 0 0 1 1 1 1 0 1 1 0 0 0 1 0
 1 0 1 1 1 0 1 1 0 0 1 0 0 0 0 1 0 0 0 1 0 1 0 0 1 0 1 0 1 0 1 1 1 0 0 1 1
 1 0 1 1 1 0 1 0 0 1 1 0 1 1 0 0 1 0 1 1 1 1 0 1 1 0 1 1 0 1 0 0 0 1 0 0 0
 0 0 0 0 1 0 0 1 1 1 1 1 1 1 0 1 0 1 1 0 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0 1 0 1 1 1 1 0 0 0 1 1
 1 1 0 1 0 1 0 1 1 1 0 1 1 1 1 1 1 1 1 0 0 0 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0 0
 0 1 0 0 1 0 1 1 1 0 1 1 1 1 1 1 1 1 1 0 1 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 1
 1 0 1 1 1 1 1 1 1 1 0 1 1 1 0 1 1 1 1 1 1 1 1 1 0 1 0 0 1 0 1 1 1 1 1 0 1 1
 0 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 0 0 1 1 1 0 1 1 0 1 1 1 1 1 1 1 1 1 1 0 1
 1 1 1 1 1 1 0 1 0 1 0 0 1 1 1 1 1 1 0 0 1 0 1 0 1 1 1 1 0 0 1 1 0 1 0 1 0 0
 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 0 1 0 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 0 0 0 0 0 0 1]

Means:
[[1.75957085e+01 2.13689346e+01 1.16319305e+02 9.91637145e+02
  1.02876168e-01 1.48130748e-01 1.63244155e-01 8.91394718e-02
  1.93444157e-01 6.29365440e-02 6.09821615e-01 1.17722062e+00
  4.34906577e+00 7.33021385e+01 6.58953275e-03 3.30124849e-02
  4.20461896e-02 1.49219947e-02 2.05045237e-02 4.13057753e-03
  2.13255680e+01 2.89627155e+01 1.42852023e+02 1.44387232e+03
  1.44874555e-01 3.86279506e-01 4.60861434e-01 1.84626613e-01
  3.26149713e-01 9.27148016e-02]
 [1.21625159e+01 1.81117822e+01 7.81751817e+01 4.64129320e+02
  9.26691857e-02 7.95351210e-02 4.66280790e-02 2.61352846e-02
  1.74204234e-01 6.27189069e-02 2.89242932e-01 1.23930447e+00
  2.02597117e+00 2.16631543e+01 7.29671246e-03 2.12101104e-02
  2.61425809e-02 1.00254128e-02 2.05636974e-02 3.60475269e-03
```

4.

PROGRAM:

```
from sklearn.datasets import make_moons
from sklearn.mixture import GaussianMixture

X, y = make_moons(
    n_samples=500,
    noise=0.08,
    random_state=42
```

)

```
gmm = GaussianMixture(n_components=2, random_state=42)
gmm.fit(X)

labels = gmm.predict(X)

print("Cluster Labels:")
print(labels)

print("\nMeans:")
print(gmm.means_)

print("\nLog-Likelihood:", gmm.score(X))
```

OUTPUT:

```
Cluster Labels:
[0 1 0 1 1 0 0 1 1 0 0 1 0 1 0 0 0 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 1 0 0 0
 1 1 1 0 1 0 1 0 1 0 0 0 0 1 1 1 0 1 1 1 0 1 1 0 1 0 0 1 1 0 1 1 0 0 0 1 1
 0 0 1 1 0 0 1 1 0 1 0 1 1 0 0 1 0 0 0 0 0 0 1 1 0 0 1 0 0 0 0 1 0 1 0 1 1
 0 0 0 0 1 1 1 1 0 1 1 0 1 1 1 0 1 1 0 0 1 0 1 1 1 1 0 0 1 1 1 0 1 1 1 0 1
 1 0 0 1 0 0 0 0 0 1 0 1 1 0 0 0 0 0 0 0 0 1 0 1 0 1 1 1 0 1 1 0 0 1 0 1 0
 1 0 1 1 1 0 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 1 0 0 1 1 0 0 1 0 0 1 0 1 0 1
 0 1 1 0 1 0 1 1 1 1 0 0 0 0 0 1 1 1 0 1 0 0 0 1 1 1 0 1 0 0 0 1 0 0 1 0 1
 0 1 1 0 1 1 1 0 1 0 1 1 1 1 0 0 0 0 0 0 1 0 1 0 1 1 0 1 0 0 0 0 1 0 0 1 1
 0 0 0 0 0 1 0 1 1 1 1 1 1 1 0 1 0 0 1 1 0 1 0 0 1 0 1 0 1 1 1 1 1 0 1 0 0
 0 1 1 0 0 1 1 1 1 0 0 1 0 0 0 1 1 1 0 0 1 0 1 1 0 1 1 1 0 1 1 1 0 1 0 1 0
 0 0 1 1 1 1 0 1 1 1 1 0 0 0 0 1 0 0 1 0 0 0 0 1 0 0 0 0 1 1 1 0 0 1 0 0 0
 0 1 1 1 1 1 0 1 0 1 1 1 0 0 0 0 1 0 1 1 1 1 0 0 1 0 0 1 0 1 1 0 0 1 0 1 1
 0 0 1 0 0 0 1 1 0 1 0 1 0 1 0 1 1 1 1 0 1 1 0 0 0 1 1 0 1 0 1 0 0 1 0 1 1
 1 0 1 0 1 0 0 1 0 1 0 0 1 1 1 1 0 0 1]

Means:
[[ 1.14544164 -0.13249802]
 [-0.13735641  0.64066725]]

Log-Likelihood: -1.737399832282144
```

5.

PROGRAM:

```
from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture

X, y = make_blobs(
    n_samples=500,
    centers=4,
    cluster_std=1.0,
    random_state=42
)

gmm = GaussianMixture(n_components=4, random_state=42)
```

```
gmm.fit(X)

labels = gmm.predict(X)

print("Cluster Labels:")
print(labels)

print("\nMeans:")
print(gmm.means_)

print("\nLog-Likelihood:", gmm.score(X))
```

OUTPUT:

```
Cluster Labels:
[1 2 0 3 2 2 1 2 0 2 0 3 0 3 2 0 3 1 1 3 0 3 0 1 1 2 2 1 1 3 2 3 3 3 2 2 0
 0 1 1 2 0 3 3 3 0 0 0 2 1 2 3 1 2 0 3 3 1 2 1 1 3 2 1 0 2 2 1 0 2 0 2 2 1
 3 1 3 2 0 3 2 0 2 3 1 1 1 1 0 3 1 2 0 2 0 1 3 0 3 1 0 0 0 1 1 3 3 1 3 1 2
 1 1 1 1 2 0 1 2 2 3 0 2 0 1 0 0 2 2 1 1 0 0 2 0 1 1 1 0 0 2 1 0 0 2 1 1 3
 3 3 2 2 0 0 3 1 3 1 2 2 1 1 0 0 2 3 0 2 1 1 2 3 3 1 1 3 3 2 2 2 3 1 3 3 1
 1 3 0 3 2 2 1 1 2 3 2 3 3 1 2 3 3 2 1 0 1 2 0 0 1 1 2 1 3 3 2 3 1 3 0 0 3
 1 3 0 3 3 2 2 0 2 0 3 2 1 2 3 2 0 0 0 2 3 0 1 1 0 3 3 2 3 3 3 3 0 0 1 3 2
 0 3 3 1 3 2 2 0 3 0 2 3 3 0 1 3 3 3 0 1 2 3 0 0 3 2 3 0 1 0 2 2 3 3 0 1 0
 1 0 2 0 2 1 0 0 3 0 2 2 0 0 2 1 0 0 0 2 1 0 0 0 1 0 0 1 1 1 2 1 1 1 1 1 3
 1 0 1 3 3 1 0 0 0 3 3 2 3 2 3 0 2 2 0 2 1 1 2 1 3 2 3 0 3 2 2 1 0 2 0 0 1
 3 2 0 2 1 0 1 0 1 1 2 1 3 2 1 2 3 2 1 2 1 0 3 3 2 2 0 3 3 3 0 1 2 3 0 0 2
 1 2 2 2 3 0 2 3 0 3 3 0 3 0 3 1 2 0 3 2 1 3 0 0 2 2 2 1 2 0 1 3 3 1 0 1 0
 3 2 2 2 3 0 0 1 3 2 2 2 0 2 3 0 1 0 2 0 3 0 3 0 2 3 1 1 3 3 1 1 3 2 0 2 0
 3 1 1 3 2 1 1 0 1 1 3 1 2 3 1 3 2 2 0]

Means:
[[ 4.72182456  1.9238556 ]
 [-8.68166179  7.45547894]
 [-7.0009649  -6.90445754]
 [-2.60242135  9.09231929]]

Log-Likelihood: -4.161992323161995
```

RESULT:

The Expectation-Maximization (EM) algorithm was successfully implemented in Python. The Gaussian Mixture Model clustered the dataset effectively, and the clustering performance was evaluated using the predicted labels and accuracy.